

Test Technique : Développeur Fullstack · Visions

Stack imposée : TypeScript · Express · MongoDB

Niveau attendu : MVP / Proof of Concept. Nous ne cherchons pas une implémentation production-ready, mais à voir comment vous appréhendez une documentation technique inconnue et comment vous en traduisez les concepts en code.

Contexte

Les Dataspaces sont des écosystèmes fédérés permettant à des organisations souveraines de partager des données de manière contrôlée, interopérable et basée sur des standards ouverts. Le Dataspace Protocol (DSP), publié sous gouvernance de l'[Eclipse Dataspace Working Group](#), est un des protocoles de référence qui définit comment des participants échangent des catalogues, négocient des contrats et initient des transferts de données.

Ce protocole s'appuie sur deux vocabulaires standards du W3C que vous allez probablement découvrir :

- DCAT v3 (*Data Catalog Vocabulary*) : pour décrire les catalogues et les jeux de données
- ODRL 2.2 (*Open Digital Rights Language*) : pour exprimer les politiques d'usage (permissions, obligations, interdictions)

Il s'agit d'un sujet niche et exigeant. Nous ne nous attendons pas à ce que vous le maîtrisiez avant ce test. Ce qui nous intéresse, c'est votre capacité à plonger dans une documentation technique dense, à en extraire les concepts essentiels et à les transcrire dans une implémentation cohérente.

Partie 1 : Compréhension du protocole

Répondez en français dans un fichier *REPONSES.md* à la racine de votre projet.

1.1 Vue d'ensemble

En vos propres mots (10 à 20 lignes maximum), expliquez :

- Ce qu'est un Dataspace et quel problème il résout
- Le rôle des deux acteurs principaux que sont le Provider et le Consumer
- Pourquoi l'interopérabilité nécessite un protocole standardisé plutôt qu'une simple API REST classique

1.2 DCAT & ODRL

Répondez brièvement aux questions suivantes :

1. Dans le DSP, qu'est-ce qu'un Catalog ? Quelle relation entretient-il avec un Dataset et une Distribution ?
2. Qu'est-ce qu'une Offer dans le contexte ODRL ? Quelle différence y a-t-il entre une Offer contenue dans un catalogue et un Agreement issu d'une négociation ?
3. Donnez un exemple concret de règle ODRL que vous pourriez exprimer pour restreindre l'usage d'un jeu de données (utilisez la structure permission, action, constraint).
4. Pourquoi les Offer dans un Catalog ou un Dataset ne doivent-elles pas contenir d'attribut target, alors que les Offer dans un ContractRequestMessage doivent en avoir un ?

1.3 Négociation de contrat

Décrivez en quelques phrases la state machine du Contract Negotiation Protocol : quels sont les états possibles, qui envoie quel message pour faire progresser l'état, et quel est l'état final qui autorise un transfert de données ?

Partie 2 : Implémentation

Implémentez un connecteur Provider minimal en TypeScript / Express. Ce connecteur doit exposer les endpoints HTTPS-binding du DSP décrits ci-dessous.

📖 Référence : [Dataspace Protocol 2025-1-RC1](#)

Sections à lire : 4 (Common Requirements), 5 & 6 (Catalog), 7 & 8 (Contract Negotiation)

2.1 Endpoints à implémenter

Métadonnées de version (section 4.3.2)

Méthode	Chemin	Description
GET	<code>/.well-known/dspace-version</code>	Retourne les versions du protocole supportées

Exemple de réponse attendue :

```
JSON
{
  "protocolVersions": [
    { "version": "2025-1", "path": "/api/v1" }
  ]
}
```

Catalog HTTPS Binding (section 6)

Méthode	Chemin	Description
POST	/catalog/request	Retourne le catalogue complet du Provider
GET	/catalog/datasets/:id	Retourne un Dataset spécifique par son identifiant

Contraintes :

- La réponse de POST /catalog/request doit être un objet **Catalog** valide contenant au moins un **Dataset** avec une **hasPolicy** (une **Offer** ODRL) et au moins une **Distribution**.
- Chaque **Offer** dans le catalogue ne doit pas contenir d'attribut **target**.
- Le champ **@context** doit inclure `"https://w3id.org/dspace/2025/1/context.jsonld"`.
- Un filtre optionnel peut être passé dans le corps de la requête (propriété **filter**).

Contract Negotiation HTTPS Binding, côté Provider (section 8.2)

Méthode	Chemin	Description
GET	/negotiations/:providerPid	Retourne l'état d'une négociation
POST	/negotiations/request	Initie une nouvelle négociation (Consumer → Provider)
POST	/negotiations/:providerPid/events	Le Consumer accepte l'offre courante (eventType : ACCEPTED)
POST	/negotiations/:providerPid/agreement/verification	Le Consumer vérifie l'accord
POST	/negotiations/:providerPid/termination	Termine une négociation

Contraintes :

- Les transitions d'état doivent respecter la state machine du protocole. Une tentative de transition invalide doit retourner un **HTTP 400** avec un **ContractNegotiationError**.
- Une négociation inexistante doit retourner **HTTP 404**.
- Le flux nominal à implémenter est le suivant :

1. Consumer → `POST /negotiations/request` → état `REQUESTED`
2. Provider accepte implicitement et passe l'état à `AGREED` (vous pouvez simuler un accord automatique côté Provider après réception)
3. Consumer → `POST /negotiations/:providerPid/events` (`ACCEPTED`) → état `ACCEPTED`
4. Consumer → `POST /negotiations/:providerPid/agreement/verification` → état `VERIFIED`
5. Provider finalise (peut être simulé) → état `FINALIZED`

Note : Les callbacks Consumer (section 8.3) sont optionnels dans le cadre de ce test.

2.2 Modèle de données

Vous remarquerez que le Dataspace Protocol ne définit pas en lui-même la couche de persistance pour la négociation et le catalogue. C'est ouvert pour l'implémentation. Vous avez le choix pour cette partie là, mais pour être conforme avec la stack technique de Visions nous recommandons d'implémenter la couche de persistance en utilisant mongodb (+ mongoose).

Partie 3 : Bonus (*optionnels*)

Ces éléments ne sont pas attendus mais valorisés :

- ☐ Un endpoint `GET /negotiations` listant toutes les négociations (hors-spec, utile pour le debug)
- ☐ Validation du body des requêtes entrantes (vérification du `@type`, des champs requis)
- ☐ Un `README.md` clair expliquant comment lancer le projet et tester les endpoints (exemples `curl` ou collection Postman/Bruno bienvenue)
- ☐ Tests unitaires ou d'intégration sur au moins un endpoint
- ☐ Implémentation des callbacks Consumer (section 8.3)
- ☐ Gestion correcte du header `Authorization` (même symbolique)

Livrables attendus

None

```
mon-projet/
├─ src/           # Code source TypeScript
├─ REPONSES.md    # Réponses à la Partie 1
├─ README.md      # Instructions de lancement
└─ package.json
```

Soumettez votre travail via un dépôt Git (GitHub, GitLab, etc.) ou une archive. Assurez-vous que le projet compile et démarre avec des commandes standards (`npm install`, `npm run dev` ou équivalent).

Une fois terminé, envoyez le lien de votre dépôt Git à l'email suivant:

- felix@visionspol.eu
- En ajoutant nicolas@visionspol.eu en CC du mail

Bonne chance ! N'hésitez pas à noter dans votre `REPONSES.md` les points du protocole que vous avez trouvés ambigus ou les choix d'implémentation que vous avez dû faire face aux zones grises de la spec.